Санкт-Петербургский государственный университет

SPD OnLine filter

Система управления нагрузкой

Никита Гребень Андрей Воронцов

Факультет прикладной математики - процессов управления

Обзор

Санкт-Петербургский государственный университет

1. Быстрая сортировка
2. Анализ алгоритма
3. Задачи

Обзор

Санкт-Петербургский государственный университет

1. Быстрая сортировка
   - Описание
   - Декомпозиция
   - Алгоритм
2. Анализ алгоритма
3. Задачи

Описание

- Алгоритм быстрой сортировки имеет в наихудшем случае для входного массива время работы $\Theta(n^2)$
- Быстрая сортировка, подобно сортировке слиянием, применяет парадигму **разделяй и властвуй**.
- Несмотря на такую работу в наихудшем случае, в среднем её сложность $\Theta(n \log n)$
- Коэффициент, скрытый при $\Theta(n \log n)$, довольно малый по величине

Декомпозиция

- **Разделение.** Массив $A[p..r]$ разбивается на два подмассива $A[p..q-1]$ и $A[q+1..r]$, таких, что каждый элемент A[p..q - 1] меньше или равен $A[q]$, который в свою очередь, не превышает любой элемент подмассива $A[q+1..r]$.
- **Властвование.** Подмассивы $A[p..q-1]$ и $A[q+1..r]$ сортируются с помощью рекурсивного вызова процедуры быстрой сортировки.
- **Комбинирование.** Поскольку массивы сортируются на месте, для их объединения не требуется никаких действий - весь массив $A[p..r]$ будет отсортированным.

Алгоритм

- Пример работы алгоритма **быстрой сортировки**.
- Существуют много способов выбора опорного элемента $A[q]$ и разбиения массива $A[p..r]$.
- Наиболее известная реализация процесса разбиения массива - процедура Хоара.
- Ниже представлен один из вариантов алгоритм быстрой сортировки, имплементирующий все компоненты декомпозиции.

Процедура сортировки


```
function QUICKSORT(A, p, r)
    if p < r then
        q = PARTITION(A, p, r)
        QUICKSORT(A, p, q − 1)
        QUICKSORT(A, q + 1, r)
    end if
end function
```

Функция разбиения массива

Санкт-Петербургский
государственный
университет

```
function PARTITION(A, p, r)
    x = A[r]
    i = p − 1
    for j = p to r − 1 do
        if A[j] ≤ x then
            i = i + 1
            Обменять A[i] и A[j]
        end if
    end for
    Обменять A[i + 1] и A[r]
return i + 1
end function
```

Алгоритм

Санкт-Петербургский
государственный
университет

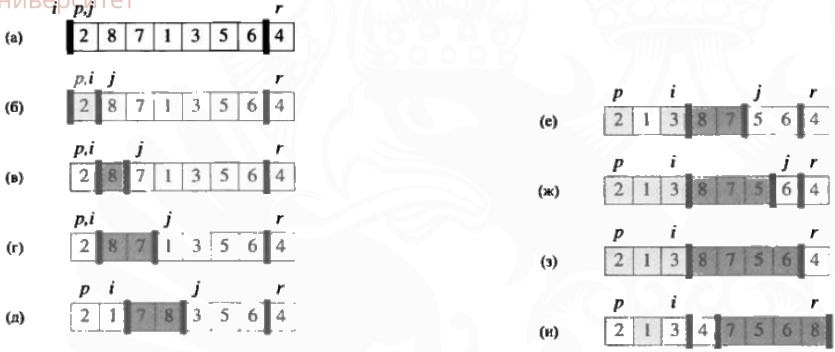

Рис. 1: Пример действия процедуры PARTITION

Алгоритм

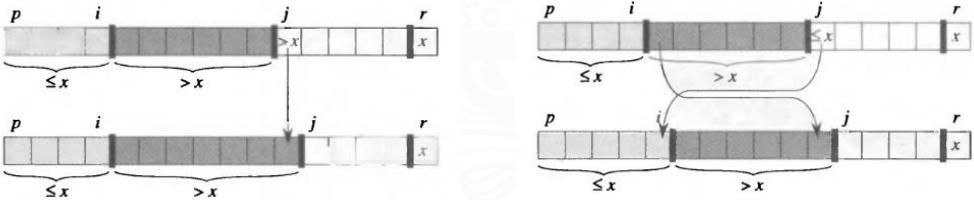

Рис. 2: Два варианта итерации в функции PARTITION

Разбиение Хоара

Санкт-Петербургский государственный университет

```
function HOARE-PARTITION(A, p, r)
    x = A[⌊(p + r)/2⌋]
    i = p, j = r
    while i ≤ j do
        while A[i] < x do
            i = i + 1
        end while
        while A[j] > x do
            j = j − 1
        end while
        if i ≥ j then break
        end if
        Обменять A[i] и A[j], i = i + 1, j = j − 1
    end while
return j
```

Обзор

Санкт-Петербургский государственный университет

1. Быстрая сортировка
2. Анализ алгоритма
   - Сбалансирование разбиение
   - Расссуждения для среднего случая
3. Задачи

Сбалансирование разбиение

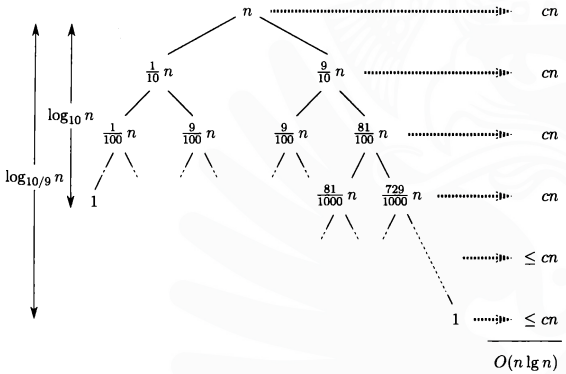

Рис. 3: Дерево рекурсии, в котором функция PARTITION всегда выполняет разбиение в соотношении девять к одному, что дает время работы $O(n \log n)$

Расссуждения для среднего случая

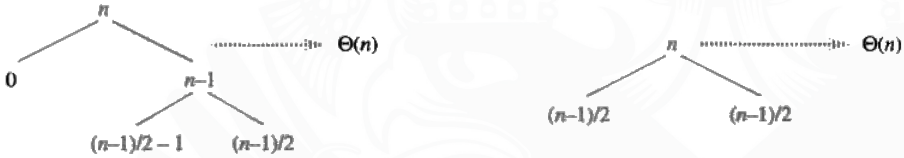

Рис. 4: Два дерева рекурсии. Левое дерево генерирует плохое разбиение на два подмассива с размерами 0 и $n-1$, что затем чередуется с хорошим разбиением на подмассивы размера $(n-1)/2$. На правом дереве представлено хорошо сбалансированное разбиение на каждом уровне рекурсии.

Обзор

Санкт-Петербургский государственный университет

1. Быстрая сортировка
2. Анализ алгоритма
3. Задачи
   - Быстрая сортировка - оптимизация (2 балла)
   - Найти k-ый элемент (3 балла)*

Быстрая сортировка - оптимизация (2 балла)

- Не трудно заметить, что в предыдуших реализациях, сортировка выполняла дальнейшие разбиения, не учитывая, что в подмассиве могли быть одинаковые значения.
- Если весь массив состоит из одинаковых значений, то он не нуждается в дальнейшем сортировке его подмассивов.
- Выполнять разбиение массива на три части: элементы которые строго меньше x, равны x, больше x.
- Опорный элемент выбирать случайно.
- Сравнить с результатом работы std::sort на массиве размера $10^7 \ldots 10^8$.

Найти k-ый элемент (3 балла)*

- Пусть $A[1..n]$ массив из n целых чисел.
- Как и в прошлом домашнем задании, найти k-ый минимальный элемент, т.е. какой элемент окажется на k-ой позиции в отсортированном массиве.
- Сложность алгоритма должна быть $O(n)$.